Library Catalog Analytics

Use the following `books` dataset for all tasks—no need to create your own data structures.

```
// Sample dataset for all exercises
const books = [
 {
 title: "The Hobbit",
 author: "Tolkien",
 year: 1937,
 rating: 4.7,
 genres: ["Fantasy"],
 },
 {
 title: "1984",
 author: "Orwell",
 year: 1949,
 rating: 4.8,
 genres: ["Dystopian", "Political Fiction"],
 },
 {
 title: "The Name of the Wind",
 author: "Rothfuss",
 year: 2007,
 rating: 4.5,
 genres: ["Fantasy", "Adventure"],
 },
 {
 title: "Brave New World",
 author: "Huxley",
 year: 1932,
 rating: 4.2,
 genres: ["Dystopian"],
 },
 {
 title: "Dune",
 author: "Herbert",
 year: 1965,
 rating: 4.6,
 genres: ["Science Fiction", "Adventure"],
 },
 {
 title: "Fahrenheit 451",
 author: "Bradbury",
 year: 1953,
 rating: 4.3,
 genres: ["Dystopian", "Science Fiction"],
 },
 {
 title: "The Road",
 author: "McCarthy",
 year: 2006,
```

```
rating: 4.0,
 genres: ["Post-Apocalyptic"],
 },
 {
 title: "To Kill a Mockingbird",
 author: "Lee",
 year: 1960,
 rating: 4.9,
 genres: ["Classic", "Coming-of-Age"],
 },
];
```

Your Tasks

1. **getRecentBooks(books, afterYear)** Return an array of titles for all books published **on or after** `afterYear`.
2. **getAverageRating(books)** Calculate and return the **average rating** of the entire collection (rounded to two decimals).
3. **sortBooksBy(books, key, asc = true)** Produce and return a **new** array of book objects sorted by the given `key` in ascending order (or descending if `asc` is `false`).
4. **countGenres(books)** Build and return an object mapping each genre to the number of books in that genre.
5. **groupByAuthor(books)** Return an object whose keys are author names and whose values are arrays of their respective book objects.
6. **hasHighlyRated(books, threshold)** Return `true` if **any** book's `rating` is at least `threshold`; otherwise return `false`.
7. **allBeforeYear(books, year)** Return `true` if **every** book's `year` is strictly before `year`; otherwise return `false`.
8. **findByTitle(books, title)** Return the **first** book object whose `title` exactly matches `title`, or `undefined` if none match.

Bonus Challenges

Tag Classics: Return a **new** array where each book object is extended with an `isClassic` boolean (`true` if `year < 1950`, else `false`).

Dystopian Titles List: Return an **alphabetical** array of titles for all books whose `genres` include `"Dystopian"`.

Keyword Search: Implement **hasKeyword(books, keyword)** to return `true` if **any** book's `title` contains `keyword` (case-insensitive).